

中山大學

SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab3-Pthreads 并行矩阵乘法与数组求和

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2026 年 4 月 29 日

1 实验目的

- Pthreads 程序编写、运行与调试
- 多线程并行矩阵乘法
- 多线程并行数组求和

2 实验内容

- 掌握 Pthreads 编程的基本流程
- 理解线程间通信与资源共享机制
- 通过性能分析明确线程数、数据规模与加速比的关系

2.1 并行矩阵乘法

- 使用 Pthreads 实现并行矩阵乘法
- 随机生成 $m \times n$ 的矩阵 A 及 $n \times k$ 的矩阵 B
- 通过多线程计算矩阵乘积 $C = A \times B$
- 调整线程数量 (1-16) 和矩阵规模 (128-2048)，记录计算时间
- 分析并行性能 (时间、效率、可扩展性)
- 实现了三种任务划分方式 (在后续实验结果分析影响)：
  - **block**：按行块划分，每个线程负责若干连续行（行数尽量均匀）。
  - **cyclic**：循环（round-robin）划分，第 t 个线程负责行 t, t+threads, ...，用于改善负载不均时的平衡。
  - **block_col**：按列块划分，每个线程负责若干连续列（将输出按列分片写入）。

2.2 并行数组求和

- 使用 Pthreads 实现并行数组求和
- 随机生成长度为 n 的整型数组 A，n 取值范围 [1M, 128M]
- 通过多线程计算数组元素和 $s = \sum_{i=1}^{n} A_i$
- 调整线程数量 (1-16) 和数组规模 (1M-128M)，记录计算时间

- 分析并行性能 (时间、效率、可扩展性)
- 实现了三种聚合策略 (在后续实验结果分析影响):
  - **serial**：每个线程计算局部和写入 **partial[tid]**，由线程 0 在最后串行合并 partial 数组得到全局和（适合线程数较小、合并开销可忽略的场景）。
  - **mutex**：每个线程在计算完局部和后通过互斥量直接把局部和累加到共享变量 **acc**（保护临界区）。
  - **atomic**：使用原子加法 **___atomic_fetch_add** 将局部和累加到共享变量，无需显式互斥。

3 实验结果

3.1 并行矩阵乘法

3.1.1 不同线程数下的运行时间

表 1: 并行矩阵乘法在不同线程数下（block 划分下）的运行时间

矩阵规模	1 线程	2 线程	4 线程	8 线程	16 线程
128×128	0.006262	0.003721	0.001869	0.000974	0.000553
256×256	0.025821	0.014999	0.011292	0.007786	0.004907
512×512	0.402789	0.150713	0.079576	0.045428	0.026021
1024×1024	2.780282	1.276648	0.643032	0.366999	0.175093
2048×2048	48.175269	23.825133	11.749670	5.884678	2.965120

分析：随线程数增加，大规模矩阵（512×512 及以上）运行时间近似按比例下降，说明计算部分占主导，具备良好的并行性；而小规模（128×128、256×256）在 8 16 线程下加速效果明显减弱，甚至趋于饱和。这表明在小问题规模下，线程创建、同步开销以及缓存一致性维护（cache coherence）成为主要瓶颈，使得并行收益被抵消。

3.1.2 加速比

为便于分析，我们计算了 block 划分下各规模在不同线程数（1,2,4,8,16）下的加速比 $S_p = T_1/T_p$ 与并行效率 $E_p = S_p/p$。

表 2: block 划分下的加速比 S_p（以 T_1/T_p 计算）

矩阵规模	S_2	S_4	S_8	S_{16}
128×128	1.68	3.35	6.43	11.33
256×256	1.72	2.29	3.32	5.26
512×512	2.67	5.06	8.87	15.48
1024×1024	2.18	4.32	7.58	15.88
2048×2048	2.02	4.10	8.19	16.24

分析：小规模（128×128、256×256）在 4-8 线程后加速比增长明显放缓，出现“扩展拐点”, 说明系统开销（线程调度、同步、缓存争用）开始主导性能；而大规模（1024×1024、2048×2048）加速比接近线性增长，表明计算密集型特征明显，能够有效利用多核资源。

3.1.3 并行效率

block 划分下的并行效率（$E_p = S_p/p$）对比：

表 3: block 划分下的并行效率 $E_p(\%)$

矩阵规模	$E_2(\%)$	$E_4(\%)$	$E_8(\%)$	$E_{16}(\%)$
128×128	84.1	83.8	80.4	70.8
256×256	86.1	57.2	41.5	32.9
512×512	133.7	126.6	110.9	96.8
1024×1024	109.0	108.1	94.7	99.3
2048×2048	101.1	102.5	102.4	101.5

分析：小规模问题并行效率较低，而在中大规模下部分测试出现 $E_p > 100\%$ 的“超线性加速”现象。这通常并非真实算法加速，而是由以下因素导致：

- CPU 动态频率提升（Turbo Boost）使多线程阶段频率更高
- 并行执行改善缓存命中率（cache effect）
- 单线程基准时间受系统干扰较大（测量噪声）

因此该现象应视为系统层优化效果，而非算法本身的线性加速突破。

3.1.4 负载均衡

程序同时输出每线程的最大计算时间 **threadMax(s)**，我们用其与整轮平均时间的比值来评价负载不均：比值越接近 1 表明线程间耗时一致。

表 4: block 划分下的 threadMax/time 比值（约）

规模	1 线程	2 线程	4 线程	8 线程	16 线程
128×128	0.995	0.991	0.980	0.944	0.848
256×256	0.999	0.997	0.995	0.993	0.981
512×512	1.000	1.000	1.000	0.999	0.998
1024×1024	1.000	1.000	1.000	0.971	1.000
2048×2048	1.000	0.999	0.998	0.999	0.998

分析：除最小规模（128×128）在高线程数下出现较大负载不均外，其它规模的比值接近 1，说明 block 划分的线程负载总体均匀。128×128 的负载不均可能由线程调度和系统干扰引起，且在小问题上线程开销占比较大，更容易受到偶发因素影响。

3.1.5 划分方式比较

下表列出三种划分（**block**/**cyclic**/**block_col**）在 1024×1024 大小下不同线程数的耗时，可用于比较数据划分对性能的影响：

表 5: 1024×1024 矩阵规模下不同划分的耗时（秒）

划分方式	1 线程	2 线程	4 线程	8 线程	16 线程
block	2.780282	1.276648	0.643032	0.366999	0.175093
cyclic	3.265369	1.614776	0.877070	0.341394	0.187485
block_col	3.165536	1.655930	0.881290	0.517020	0.252884

分析：在本机测试中 **block** 划分在多数线程数下表现最好（总体耗时最低）；**cyclic** 在某些线程数下略优于 **block_col**，但三者差异随线程数和问题规模变化而改变，说明划分方式与缓存局部性、内存带宽以及线程负载平衡共同影响性能。

为直接比较不同划分的并行表现，我们计算了每种划分在 1024×1024 下相对于其单线程时间的加速比：

表 6: 1024×1024，三种划分的加速比 S_p

划分	S_2	S_4	S_8	S_{16}
block	2.18	4.32	7.58	15.88
cyclic	2.02	3.72	9.57	17.42
block_col	1.91	3.59	6.12	12.52

分析：对比三种划分可见，**block** 在多数线程配置下表现稳健；**cyclic** 在 8 线程时

对该实现有显著提升，表明细粒度循环划分在特定并发度下能改善局部负载与内存通路利用。

3.2 并行数组求和

3.2.1 不同规模与线程数下的运行时间

表 7: 数组求和（partition=block, agg=serial）不同规模与线程数下的运行时间

数组规模	1 线程	2 线程	4 线程	8 线程	16 线程
1M	0.001500	0.000781	0.000421	0.000257	0.000220
4M	0.005113	0.003042	0.001586	0.000829	0.000514
16M	0.015014	0.010193	0.006261	0.003887	0.001737
64M	0.047326	0.025810	0.015357	0.010748	0.006210
128M	0.090309	0.054463	0.026667	0.016444	0.024920

分析：运行时间整体随线程数增加而下降，但在 128M 且 16 线程时出现性能回退。这表明程序从“计算受限（compute-bound）”逐渐转变为“内存带宽受限（memory-bound）”。当线程数过多时，多核同时访问主存导致带宽竞争加剧，使得额外线程无法带来性能提升，反而增加调度与同步开销。

3.2.2 加速比

block 划分下的加速比（以 T_1/T_p 计算）对比分析

表 8: block 划分下的加速比（以 T_1/T_p 计算）

数组规模	2 线程	4 线程	8 线程	16 线程
1M	1.92	3.56	5.84	6.82
4M	1.68	3.22	6.17	9.95
16M	1.47	2.40	3.86	8.64
64M	1.83	3.08	4.40	7.62
128M	1.66	3.39	5.49	3.62

分析：加速比随规模增加总体提升；128M 在 16 线程下加速比回落，提示聚合阶段或内存带宽限制导致扩展性受限。

3.2.3 并行效率

在 block 划分下的并行效率（%）分析

表 9: block 划分下的并行效率（%）

数组规模	2 线程 (%)	4 线程 (%)	8 线程 (%)	16 线程 (%)
1M	96.0	89.0	73.0	42.6
4M	84.0	80.6	77.1	62.2
16M	73.6	60.0	48.3	54.0
64M	91.7	77.0	55.0	47.6
128M	83.0	84.7	68.7	22.7

分析：并行效率在中小规模随线程增多逐步下降；64M/128M 显示内存子系统与聚合策略对效率有显著影响，建议使用 **atomic** 或分层归约以减小合并瓶颈。

3.2.4 聚合方式比较

下表比较了在 128M 数据上，不同聚合策略（serial/mutex/atomic）随线程数的耗时：

表 10: 128M，聚合方式比较（秒）

threads	serial	mutex	atomic
1	0.090309	0.091985	0.090025
2	0.054463	0.052500	0.048191
4	0.026667	0.033250	0.026287
8	0.016444	0.041586	0.022340
16	0.024920	0.011697	0.009956

分析：聚合策略对高线程数性能影响显著：

- **atomic**：采用硬件级原子操作，避免显式锁竞争，适合高并发场景
- **mutex**：存在锁竞争与上下文切换开销，在高线程数下性能下降
- **serial**：最终合并阶段完全串行，成为明显瓶颈（Amdahl 定律体现）

实验结果表明，在 16 线程下 **atomic** 最优，验证了细粒度同步优于粗粒度锁机制。

4 实验分析

4.1 并行矩阵乘法

- **线程数量对性能的影响分析**：

对于小规模矩阵（如 128×128、256×256），随线程数增加并行效率明显下降，说明线程创建、调度与同步开销在总执行时间中占比较高。此外，小规模数据难以充分利用缓存层次结构，多线程反而加剧缓存一致性（cache coherence）开销。

对于大规模问题（ 512×512），计算密度显著提高，线程间计算任务足够大，调度开销被摊薄，从而表现出接近线性的加速趋势。但当线程数继续增加（如 16 线程），性能提升开始趋缓，说明系统逐渐受到内存带宽与缓存竞争的限制。

- **矩阵规模对并行效率的影响**：

  随着矩阵规模增大，计算复杂度（$O(n^3)$）增长速度远高于线程管理开销，使得并行计算占比提高，从而显著提升并行效率。

  在中大规模（1024×1024、2048×2048）下，实验结果显示加速比接近线性，说明程序具有良好的计算密集型特征。同时，部分测试中出现 $E_p > 100\%$ 的现象，这通常由缓存命中率提升或 CPU 动态频率调整（Turbo Boost）引起，而非真正突破理论加速上限。

  总体来看，问题规模越大，并行化收益越稳定，系统更接近理想并行模型。

- **可扩展性分析**：

  从 2048×2048 的实验结果可以看出，加速比在 16 线程时仍接近线性增长，说明该实现具有较好的强可扩展性（Strong Scaling）。

  然而，从效率变化趋势可以观察到，当线程数增加时，性能提升逐渐受到以下因素限制：

  – 内存带宽瓶颈（memory bandwidth saturation）
  – 多线程缓存竞争（cache contention）
  – NUMA 架构下的跨节点访问开销（若存在）

  因此，在更高线程数下（如 >16），若不进行优化（如缓存分块 tiling、NUMA 绑定），性能提升将进一步受限。

- **不同数据划分方式的比较**：

  以 1024×1024 为例，**block** 划分在多数线程配置下表现最佳，其原因在于：

  – 连续行访问保证了良好的缓存局部性（spatial locality）
  – 减少跨线程的数据共享，降低 cache coherence 开销

  cyclic 划分通过打散任务，在某些线程数（如 8 线程）下能够改善轻微负载不均问题，但由于访问模式不连续，会降低缓存命中率，因此整体表现略不稳定。

block_col 由于按列划分，访问矩阵时跨行跳跃（stride 访问），严重破坏缓存局部性，因此性能最差。

可以说明在计算密集型问题中，**缓存友好性往往比负载均衡更关键**。

4.2 并行数组求和

- **线程数量对性能的影响分析**：

  对于小规模（1M、4M），线程创建与同步开销占比较高，导致加速比增长受限，属于典型的“并行开销主导”场景。

  对于中大规模（16M 及以上），计算占比提高，多线程带来明显性能提升。但在 128M 且 16 线程时出现性能回退，表明系统进入“内存带宽受限”（memory-bound）阶段。

  此时，增加线程数不会提升性能，反而由于：

  - 内存访问冲突
  - cache line 争用（false sharing 可能存在）

  导致整体效率下降。

- **数组规模对并行效率的影响**：

  随着数组规模增大，计算任务变得更加均匀且计算占比提高，使并行效率整体提升。

  但由于该问题本质为低计算强度（memory-bound），性能最终受限于内存带宽而非计算能力。因此在大规模（如 128M）时，即使增加线程数，也无法持续获得线性加速，甚至出现效率下降现象。

  这体现了典型的 Roofline Model 特征：性能上限由内存带宽决定。

- **可扩展性分析**：

  从加速比曲线可以看出，该程序在 8 线程之前具有较好的扩展性，但在 16 线程时出现明显性能下降，说明扩展性受到限制。

  主要瓶颈包括：

  - 内存带宽饱和（bandwidth saturation）
  - 聚合阶段串行化（尤其是 serial 模式）
  - 原子操作或锁竞争带来的同步开销

  因此，该程序属于“弱扩展性受限”类型，优化重点应放在减少内存访问和改进归约方式。

- **不同聚合方式的比较**：

  实验结果表明，不同聚合策略对高线程数性能影响显著：

  - **atomic**：采用硬件级原子操作，避免显式锁竞争，具有较低同步开销，在高并发下表现最好
  - **mutex**：需要加锁与解锁，产生线程阻塞与上下文切换，性能随线程数增加明显下降
  - **serial**：最终合并阶段完全串行，符合 Amdahl 定律中的串行瓶颈，严重限制扩展性

  实验中 16 线程时 **atomic** 表现最优，说明细粒度同步（atomic）优于粗粒度锁（mutex），远优于串行归约。同时也说明，在高并发程序中，**归约策略往往比计算本身更关键**。